

Fast parallel Newton–Raphson power flow solver for large number of system calculations with CPU and GPU

Zhenqi Wang^{*}, Sebastian Wende-von Berg, Martin Braun

Department of Energy Management and Power System Operation, University of Kassel, Wilhelmshöher Allee 73, Kassel, 34121, Germany
Fraunhofer Institute for Energy Economics and Energy System Technology, Königstor 59, 34119 Kassel, Germany



ARTICLE INFO

Article history:

Received 14 July 2020

Received in revised form 2 March 2021

Accepted 17 April 2021

Available online 24 April 2021

Keywords:

Probabilistic Power Flow

Monte Carlo Simulation

Contingency analysis

GPU-acceleration

Newton–Raphson

Parallel computing

ABSTRACT

To analyze large sets of grid states, e.g. when evaluating the impact from the uncertainties of the renewable generation with probabilistic Monte Carlo simulation or in stationary time series simulation, large number of power flow calculations have to be performed. For the application in real-time grid operation, grid planning and in further cases when computational time is critical, a novel approach on simultaneous parallelization of many Newton–Raphson power flow calculations on CPU and with GPU-acceleration is proposed. The result shows a speed-up of over $\times 100$ comparing to the open-source tool pandapower, when performing repetitive power flows of system with admittance matrix of the same sparsity pattern on both CPU and GPU. The speed-up relies on the algorithm improvement and highly optimized parallelization strategy, which can reduce the repetitive work and saturate the high hardware computational capability of modern CPUs and GPUs well. This is achieved with the proposed batched sparse matrix operation and batched linear solver based on LU-refactorization. The batched linear solver shows a large performance improvement comparing to the state-of-the-art linear system solver KLU library and a better saturation of the GPU performance with small problem scale. Finally, the method of integrating the proposed solver into pandapower is presented, thus the parallel power flow solver with outstanding performance can be easily applied in challenging real-life grid operation and innovative researches e.g. data-driven machine learning studies.

© 2021 Elsevier Ltd. All rights reserved.

1. Introduction

The penetration of Distributed Energy Resources (DER) e.g. Wind and PV causes high uncertainties in planning and operation of power systems. For both grid planning and operation, with the probability distribution of the infeed and load with uncertainties known, it can be formulated as Probabilistic Power Flow (PPF)-problem and evaluated with Monte Carlo Simulation (MCS)-PF. The computational effort to solve the PPF problem varies according to the complexity of the unknown variable and the sampling methods. In the past decades, the researches have succeeded in reducing the required number of PFs [1–3]. Besides, the MCS-PF finds good application on a similar evaluation for the uncertainty with historical or synthetic injection and load profiles.

Static N-1 contingency analysis is required for the real-time and future grid state in the grid operation, with which grid congestion caused by the renewable energy needs to be properly

handled with market and operational measurements [4]. Thus, it is essential to evaluate large number of PFs fast.

Furthermore, recent power system research trends show an increasing demand for a powerful PF solver for many PFs. Data-driven machine learning methods show great potential in power system state estimation, approximation of PF and using Artificial Neural Network (ANN) to assist decision making in dispatch actions in grid operation. Those approaches require large number of PFs of similar grid topology to be performed in the training phase [4–6]. The proposed method finds successful application in the study of using ANN for Medium-Voltage (MV) power system state estimation [7].

With the open source tools e.g. MATPOWER [8] and pandapower [9], the grid can be easily modeled and single PF can be conveniently solved. The performance of these tools in solving many PFs with similar grid topology is unsatisfactory. Emerging works have shown great potential of the application of massive parallel hardware e.g. GPU in acceleration single PF [10–13] as well as many PFs e.g. for static contingency analysis and online PPF [14–19].

Motivated by the aforementioned challenging use cases, a general-purpose parallel PF solver for solving many PFs for grid with same sparse pattern of admittance matrix is presented in

^{*} Correspondence to: University of Kassel, Department of Electrical Engineering and Computer Science, Department of Energy Management and Power System Operation, Germany.

E-mail address: zhenqi.wang@uni-kassel.de (Z. Wang).

this work. PF with classical NR method is efficiently solved on both CPU and GPU with the proposed approach. Recent development [15,20,21] show the advantage of parallelization for solving many PFs through batched operation on GPU. Our work distinguished itself by furthering to study the possible bottlenecks in the NR algorithm. Through further optimization, the repetitive work can be reduced and the utilization rate of available computational resources can be maximized.

The main contributions of our work include the following. First, to optimize the performance on CPU, besides the task level parallelization on CPU, a new parallelization scheme on CPU with the extra explicit Single Instruction Multiple Data (SIMD) parallelization [22], which is the first of its kind to our best knowledge as revealed in our literature overview. The multi-threaded SIMD LU refactorization shows further speed up comparing to the state-of-the-art KLU [23] library with task-level parallelization. Secondly, an easy-to-implement row-level parallelization strategy is proposed for batched sparse matrix operation on GPU. Thirdly, an improved GPU LU refactorization based on the latest advances [20,24] is proposed, which can increase the hardware saturation through the final stages in the LU-refactorization process and thus improve the performance on small batch size. Furthermore, the forward substitution backward substitution step is optimized with fine-grained parallelization strategy. Last but not least, the method of integrating the parallel PF solver into the python-based open-source power system analysis tool is presented, which is essential of the application into real-life grid planning, operation and researches.

This paper is formulated in 5 sections. Section 2 introduces the CPU and GPU architecture and how the performance of computation tasks can be optimized respectively. Section 3 introduces the proposed approach of implementation of the parallel PF solver. Section 4 introduces the proposed batched linear solver for CPU and GPU platform. Section 5 presents the benchmarking result with comprehensive analysis.

2. Review of parallelization on CPU and GPU

This section gives an overview of the modern CPU and GPU regarding its specialties in order to solve many PFs efficiently. Fig. 1 shows a generalized CPU–GPU hardware structure. On modern CPU, multiple physical cores are usually available, with which multiple tasks can be performed independently. For computational tasks, a good utilization of multi-cores greatly improves the performance. Furthermore, with large size of on-chip cache available, the memory transaction with limited memory bandwidth can be avoided on cache-hits.

The SIMD instruction set is available on CPU on each physical core, which can execute the same operation on multiple data resides in the extended registers with the size of 128 bits of higher [25]. This corresponds to 2 and more double precision FPs, with which the overall FP capability of CPU is greatly improved [26]. The SIMD instruction set e.g. SSE and AVX2 is available on most modern CPU, which can carry out 2 and 4 double-precision FPs simultaneously.

Comparing to CPU, modern GPU is designed not only for graphic rendering but also as a powerful highly parallel programmable processor [27]. Large number of parallel Stream Processor (SP)s are integrated on-chip, which result in a high FPs peak performance comparing to CPU. The SPs are clustered into Stream Multiprocessor (SM)s, which contains L1-cache, shared memory and control units [28]. Besides, GPU has high memory bandwidth as shown in Fig. 1, which brings advantage on the memory-bounding tasks.

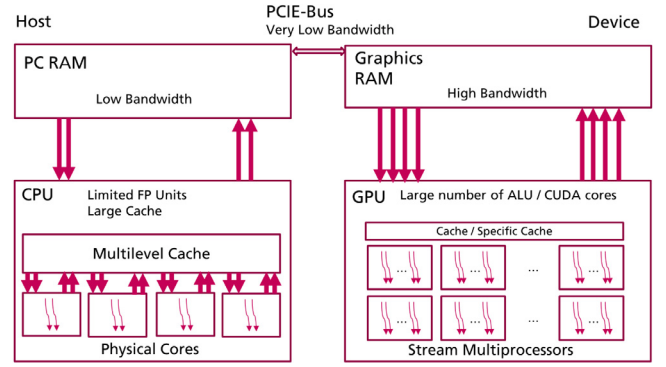


Fig. 1. CPU–GPU architecture overview.

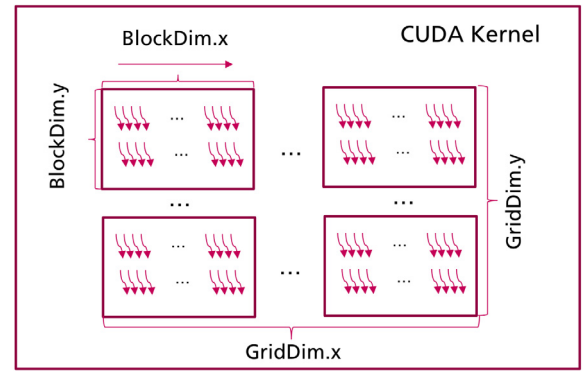


Fig. 2. Simplified model of CUDA kernel structure.

2.1. Parallelization on CPU

Multi-threaded task-level parallelization can be realized with OpenMP [29], which is a popular library in scientific computing for parallelization on shared-memory. This parallelism paradigm is called Simultaneous multithreading (SMT) [30]. OpenMP uses compiler directives to allocate the parallelized regions in the serialized program during the compiling phase.

Since OpenMP 4.0, SIMD parallelization is directly supported within the framework [31]. On top of the SMT parallelization, mini-batch tasks can further profit from the usage of SIMD instruction set, this type of parallelism is denoted as SMT+SIMD.

2.2. Principle of SIMT parallelization on GPU

In order to perform large-scaling scientific computation tasks on massive parallelization hardware like GPU, Single Instruction Multiple Threads (SIMT) programming model is widely adapted. Using NVIDIA CUDA as an example, the user is able to program functions called CUDA Kernel, which defines the behavior of massive simultaneous threads. A simplified 2D programming model of CUDA kernel is shown in Fig. 2. The massive threads are organized in a two-level structure, with each level supports up to 3-dimensions. The first level is called thread block, all the threads within a block are executed on a single SM and can be synchronized explicitly with synchronization barrier [28]. All the thread blocks are organized in a grid. The execution of grid is distributed on all the SMs on-chip. The thread is aware of its indexing in the two-level structure, which is essential to define the data-indexing relevant individual tasks. The size of thread block and grid can be configured upon kernel call.

During the execution of a CUDA kernel, 32 consecutive threads within one block are aggregated into the minimum execution unit

called warp [32]. The threads within one warp execute the same instruction. When threads within a warp encounter different instructions, these are executed in a serial mode, thus lose the advantage of parallelism. To fully utilize the calculation capability for computing-bounding tasks, the first criterion for GPU performance optimization is to avoid thread divergence within one warp.

Optimizing memory operation with the on-device memory is another important factor. Due to the high-latency of the graphics memory (multiple hundreds of cycles) [33], the instruction of memory transaction is queued in a First-In-First-Out way. It is possible to hide this latency and saturate the memory bandwidth by keeping the queue busy [32]. For each memory transaction, 32-bytes data of consecutive memory sections is accessed, to maximize the efficiency of memory transaction and reduce excessive memory transaction, it is important to enable the warp access for coalesced memory, especially for memory-bounding tasks [34]. Constants and texture can be accessed through specific cache, which improves the access with sparse matrix indexing.

Any tasks on GPU can only be executed, when the data is available on the device memory. Fig. 1 shows the limited bandwidth between CPU (Host, H) and GPU (Device, D). To optimize this, the required data transaction between host and device should be possibly reduced. Furthermore, CUDA offered the possibility of overlapping different operations (H2D, D2H memory transaction and kernel execution) through the usage of CUDA stream [19].

3. Approach for a parallel NR-PF solver

In this work, an approach of the acceleration of many NR-PFs with same sparsity pattern admittance matrix Y_{bus} (according to the MATPOWER convention [8]) is presented. This special property leads to the same sparsity pattern of the update jacobian matrix (JM) required in the iterative solving process, which brings further advantages for speed-up by reducing repetitive works as following:

- Reuse of indexing of sparse matrix
- Reuse of static lookup for sparse matrix update
- Reuse of memory working space

Fig. 3 shows exemplary the indexing data and profile data required for the parallel PF solver. The Y_{bus} is stored in Compressed Row Storage (CRS)-format with extended diagonal index. The standard CRS sparse matrix indexing consists of *RowPtr* with the length of number of rows plus one and *ColIx* equals the number of non-zero elements, which is efficient for the iteration over rows of the matrix. The extra *DiagPtr* to represent the data index of the diagonal element of each row for the convenience of iterating and aggregating on the diagonal elements (required in update JM). Fig. 3b gives the aforementioned sparse indexing of the non-zero elements of Y_{bus} in Fig. 3a. For different types of calculation, the variable profile data required is shown in Fig. 3c. For PPF, Y_{bus} requires only once while the PQ equals the size of the sampling. For static N-1 analysis, the number of Y_{bus} equals the number of contingency cases (number of post-contingency matrices) while PQ requires only once.

$$S_{bus} = V_{bus} \cdot (Y_{bus} \cdot V_{bus})^* \quad (1)$$

As shown in Fig. 3a and given in Eq. (1), PF problem is defined as finding the unknown complex voltage V_{bus} , which minimized the power injection mismatch between the given complex bus injection S_{bus} and the one calculated with V_{bus} [35]. With different bus types, different values are given as input. For slack bus the complex voltage is given, while for PV bus the constant voltage magnitude and active power and for the PQ bus with active and reactive power is predefined (shown in Fig. 3a). The

complex voltage is represented in polar notation ($\angle V, |V|$) and complex power injection S_{bus} is given in cartesian form (P, Q). For NR algorithm, the following processes are performed iteratively until maximum allowed iteration number reached or the maximum of the power injection mismatch drops below the tolerance (typically 10^{-8} p.u.[8]).

1. Calculate Nodal Power Mismatch (NPM) $\Delta P_V, \Delta Q_V$ with V_{bus}
2. Create/Update JM J with V_{bus}
3. Solve the linear system
4. Update V_{bus} with $\Delta \angle V_{pv,pq}, \Delta |V_{pq}|$

In each iteration, the convergence (power injection mismatch) is checked with the NPM calculation. Section 4 explained the details of solving linear system step with LU-refactorization.

3.1. Batched operation on sparse matrix

The utilization of the sparse matrix is essential to reduce the required memory space and avoid useless arithmetic operations on zeros, due to the high sparsity rate in Y_{bus}, J . Since the sparse matrices Y_{bus}, J share the same sparsity pattern, when iterating over the elements, it is possible to broadcast operations performed on these matrices among tasks within batch. The storage of the batched profile data in memory is shown in Fig. 3(d). The batched profile data in memory is aligned on the same element within the batch (mini-batch) and stored in contiguous memory address instead of within each task vector. On GPU, this property guarantees automatically coalesced memory access and avoid the threads divergence within one warp [20], as long as the batch has a size of multiply of the default warp size (32 for CUDA). With SMT+SIMD on CPU with mini-batch, it makes full utilization of the FP capability of the instruction set. A mini-batch of size 4 is used for the efficient usage of AVX2-instruction set.

In order to achieve an early saturation of the GPU performance or memory bandwidth depends on the property of the kernel. Further actions to increase the saturation is required, as explained below.

3.1.1. Calculate nodal power injection mismatch

Algorithm 1 Batched calculation of NPM

```

1: # Do in parallel on CUDA
2: # Find taskId (tId) with thread indexing
3: row = ThreadBlock.Idx
4: batchId = ThreadBlock.Idx
5: tIdinBatch = Thread.Idx
6: tId = batchId * ThreadBlock.DimX + tIdinBatch
7: # Initialize P, Q mismatch vector
8:  $\Delta P_V(row, tId) = -P_0(row, tId)$ 
9:  $\Delta Q_V(row, tId) = -Q_0(row, tId)$ 
10: # Calculate nodal power injection
11: for dId in RowPtr(row : row + 1) do
12:   col = ColIx(dataIx)
13:    $|S| = |V(row, tId)| \cdot |Y(dId, tId)| \cdot |V(col, tId)|$ 
14:    $\angle S = \angle V(row, tId) - \angle Y(dId, tId) - \angle V(col, tId)$ 
15:   # Update P, Q mismatch
16:   if row in buspv,pq then
17:      $\Delta P_V(row, tId) += |S| \cdot \cos(\angle S)$ 
18:   end if
19:   if row in buspq then
20:      $\Delta Q_V(row, tId) += |S| \cdot \sin(\angle S)$ 
21:   end if
22: end for

```

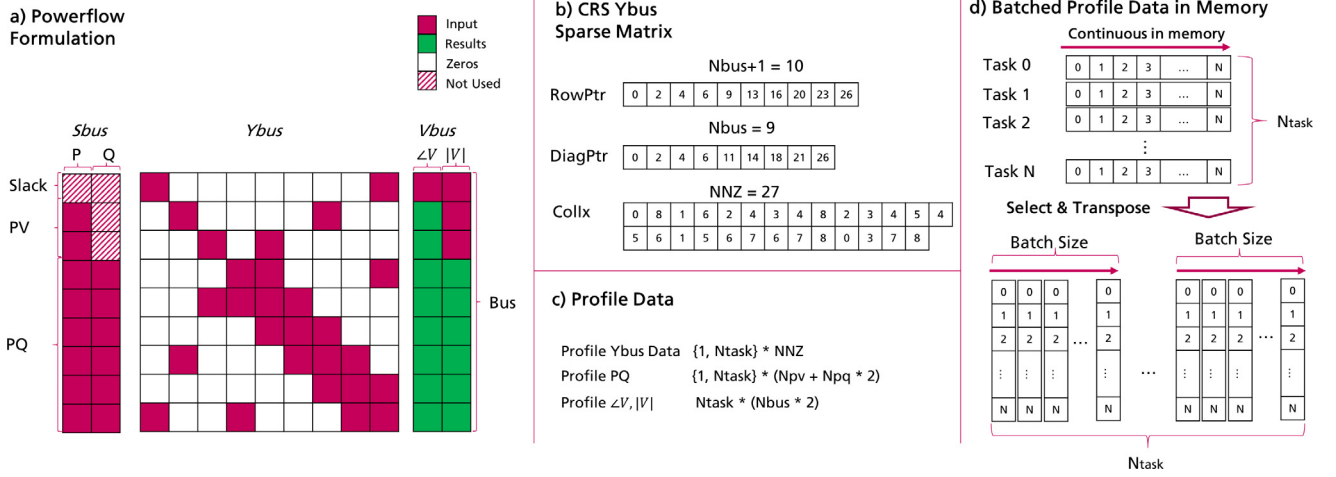


Fig. 3. CRS sparse matrix for NR-PF and batched data.

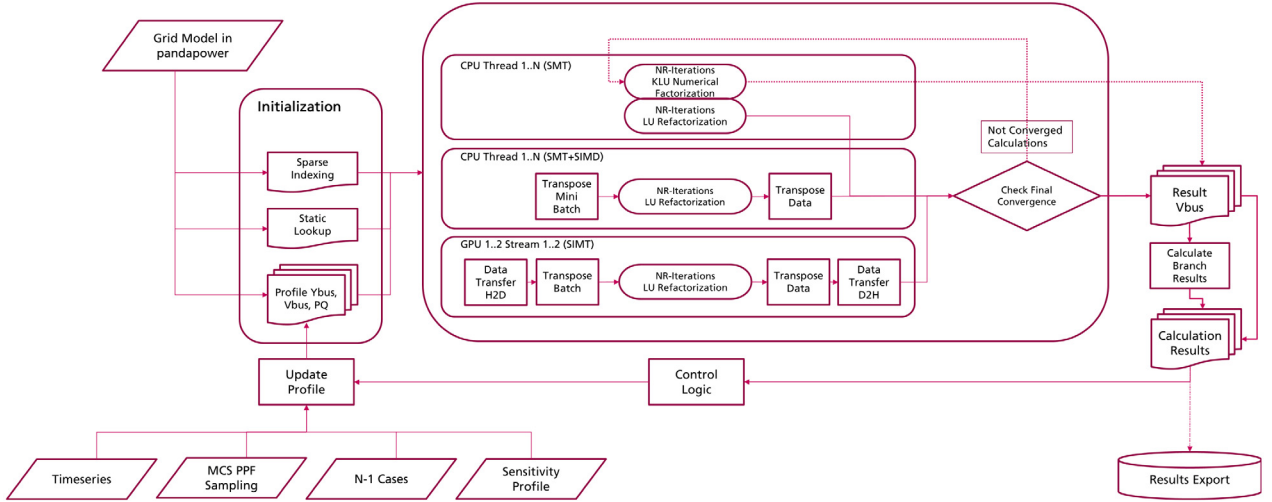


Fig. 4. System overview of parallel PF solver.

Calculation of NPM with V_{bus} is given in Eq. (1). The process can be performed with iterating over the element in the Y_{bus} once. Due to the task independency between rows in Y_{bus} , extra row-level parallelism can be achieved with one CUDA thread responsible for a row of one task. Algorithm 1 shows the proposed process on GPU, which is a computing-bounding task, the extra row-level parallelization improves the kernel saturation on GPU. On CPU platform, with SMT parallelization the $taskId$ is given as an input. For SMT+SIMD parallelization the arithmetic operation is automatically broadcasted to all the task within the mini-batch.

3.1.2. Update Jacobian matrix

Similar to Algorithm 1, update of J matrix is a computing-bounding task. Due to the independency between rows, the process can be extended with row-level parallelization for a better saturation.

In the first step, only the diagonal elements are calculated with help of the $DiagPtr$ as shown in Fig. 3b. On the second step, the non-diagonal elements are calculated by iterating over the Y_{bus} once, during which the diagonal elements are updated. With the $d\{P, Q\}/d\{\angle V, |V|\}$ matrix consisting of four sub-matrices available, it can be subset into J matrix (without P, Q for slack and Q for PV bus) or direct into permuted form A with static lookup.

3.2. System overview of parallel PF solver and its application

Fig. 4 shows the method of integrating the aforementioned parallel PF solver with pandapower. The initialization step is carried out with the pandapower in python environment including the initialization of the sparse indexing of Y_{bus} and J as well as the static lookup for updating sparse matrix and the profile data. With unified initialization step, the PFs can be solved with the three types (SMT, SMT+SIMD, GPU SIMT) of parallel PF solver. The SMT+SIMD requires the extra transpose to mini-batch step, while the GPU version requires the memory transaction between device and host and the transpose to batch step on device. The resulted power injection mismatch is checked on CPU, if any PF task cannot converge due to numerical instability during LU refactorization, this task will be given a second chance to get fixed with the KLU numerical factorization. The branch flow is calculated in parallel after the final convergence check with the resulted V_{bus} .

The proposed method, as indicated by its parallel nature of performing task-level PFs under the circumstances that Y_{bus} and J remains the same sparsity pattern, can profit at largest, when no dependency needs to be considered between PFs. This kind of use cases include PPF, N-1 analysis, stationary time-series analysis without switch configuration change and some training processes in machine learning. For use case like quasi-static time-series

Table 1
Benchmark results with separately listed timing of the initialization.

Case name	case118	case300	case1354	case2869	case9241	sb mv-lv	sb hv-mv	sb ehv-hv
Number of buses	118	300	1354	2869	9241	115	1787	713
Number of branches	186	411	1991	4582	16019	115	1836	1275
Timing of the initialization (ms)								
	24	67	267	608	2211	36	353	142
Timing of 10,000 PFs (ms)								
Pandapower	18,956	47,373	149,234	464,009	1,794,955	12,366	126,062	101,279
SMT best	175	778	2,666	8,323	40,835	107	2,291	1,558
SMT+SIMD best	97	333	1,524	7,217	38,848	52	1,646	721
1 gpu best	50	201	639	1,941	10,658	43	656	449
2 gpu best	26	102	342	1,021	6,525	24	353	233

simulation in distribution grid, in which the discrete actions such as On-Load Tap Changer (OLTC) tap position, status of shunt compensator rely on the status continuity but has no impact on the sparsity pattern. This type of simulation can still be accelerated by the proposed method, with an external control logic to update the profile with the persistence of the continuity of discrete variable. However, the control logic might lead to extra loops to correct the excessive status changes, which impacts negatively on the computational efficiency. For simulations with topology changes, such as switch position or status change, the extra initialization (costly, as shown in Table 1) needs to be performed for each new topology. When only limited variations of grid configuration needs to be considered, the proposed method can still accelerate the overall simulation.

The proposed approach is realized with C++/CUDA with an interface to Python with Cython 0.29.21[36]. The programming language Python has convenient tool for data integration and processing e.g. Pandas (High-Level) [37] and efficient numerical operations with Numpy [38]. The underlying data stored with Numpy array stored in C format can be shared and ported directly to the parallel PF solver.

4. Batched linear system solver

The solving of linear system is a time-consuming step in solving NR PF, as it is in the actual pandapower implementation [39]. The mathematical formulation of solving linear system step in NR process is given in Eq. (2). The b is the active and reactive power mismatch with the given V , the result x is used to update the V .

$$\begin{aligned}
 J_{1..n} \cdot x_{1..n} &= b_{1..n} \\
 x &= [\Delta \angle V_{pv,pq}, \Delta |V_{pq}|] \\
 b &= [\Delta P_V, \Delta Q_V]
 \end{aligned} \quad (2)$$

In this work, the existing process of solving linear system is analyzed and an optimized algorithm for batched linear system solver is proposed.

4.1. Process of solving with direct LU solver

In solving linear system with LU factorization, the original linear system is pre-ordered (pre-scaled), factorized into lower-triangular matrix L and upper-triangular matrix U and finally solved with FS-BS, as it is in the modern implementations [23,24,40,41]. KLU is considered one of the fastest single-thread linear solver for JM [12,16,20,42], which is used as the reference for the proposed method in this work. The KLU solver can be easily extended for task-level parallelization with OpenMP. Following theoretical introduction focuses on the implementation of KLU and gives hints on the optimization for solving many PFs with same Y_{bus} sparsity pattern.

In the pre-ordering phase, the original J is permuted in order to reduce the required FPs by reducing the fill-ins. Multiple

heuristic methods are available for the pre-ordering, which gives different performance related to the matrix pattern. In this work, the pre-ordering method (AMD [43]) available in KLU is used, since [19,42] reported its good performance in reducing fill-ins generally on circuit simulation and PF analysis with NR algorithm. In other cases, such as to factorize augmented JM to directly consider control effect from e.g. OLTC, the work [44] presented an extra analysis regarding the pre-ordering method. With the permutation, the original linear system J is permuted into the A as given in Eq. (3). $Perm_{col}$, $Perm_{row}$ are the correspondent column and row permutation matrix. The numerical factorization is performed on A .

$$\begin{aligned}
 A_{1..n} &= Perm_{col} \cdot J_{1..n} \cdot Perm_{row} \\
 A_{1..n} &= L_{1..n} \cdot U_{1..n}
 \end{aligned} \quad (3)$$

In the numerical factorization of KLU, the Gilbert-Peierls left-looking algorithm G-P algorithm is utilized. Additionally, partial pivoting is performed to improve the numerical stability, with which the permutation matrix $Perm_{row}$ is updated, in order to avoid the tiny pivot. This step has effect on the final sparsity pattern on A .

Refactorization is a much faster process, which reduced the computational overhead by presuming the numerical stability of the permutation matrix $Perm_{row}$, $Perm_{col}$. For NR iterations in solving PF and circuit simulation, refactorization is preferred [24]. The refactorization mode is also supported by KLU.

The pattern of A , L and U remains unchanged for all tasks when solving many PFs with same Y_{bus} sparsity pattern. The pre-ordering only need to be performed once at the beginning [15]. Based on this property, a static lookup can be created with the final permutation matrices $Perm_{row}$, $Perm_{col}$ found. Since in G-P algorithm Compressed Column Storage (CCS) matrix is required, which is efficient for iterating over columns. A static lookup can direct convert the original CRS JM or $d\{P, Q\}/d\{\angle V, |V|\}$ matrix into the permuted A into CCS format.

4.2. CPU LU refactorization

Algorithm 2 gives the SMT LU refactorization algorithm. Sparse G-P[45] refactorization on CPU with column working space is implemented. The working space has the size of the dimension of A for SMT. With the column working space x , only the copy of non-zero values of A is required for the sparse vector multiply and add (VMAD).

For SMT+SIMD, working space with the width of the mini-batch size is required. By replacing $tlid$ with the ID of mini-batch, the copy, VMAD and normalization can be extended with SIMD instructions, so that the operation is broadcasted within mini-batch.

Since task-level parallelization can fully saturate all physical cores of CPU with SMT and SMT+SIMD, column-level parallelization as proposed in [46] is not needed.

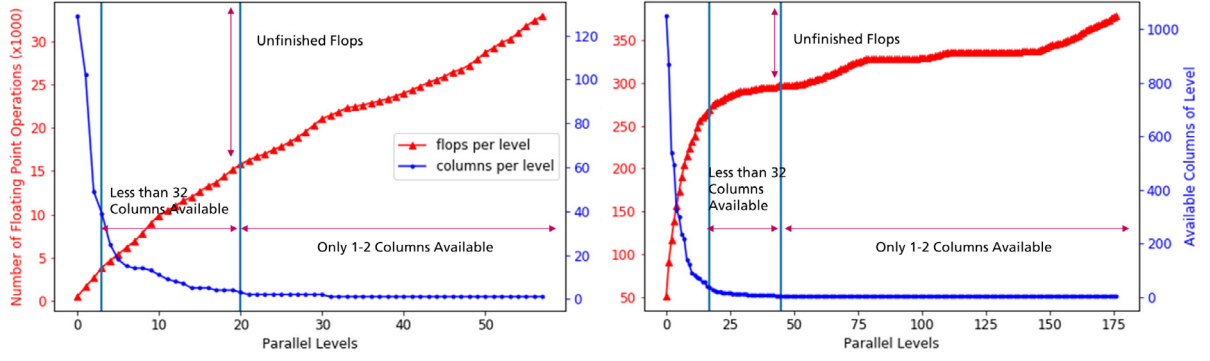


Fig. 5. LU Parallel Levels and FPs (left: IEEE case300, right: case2869pegase).

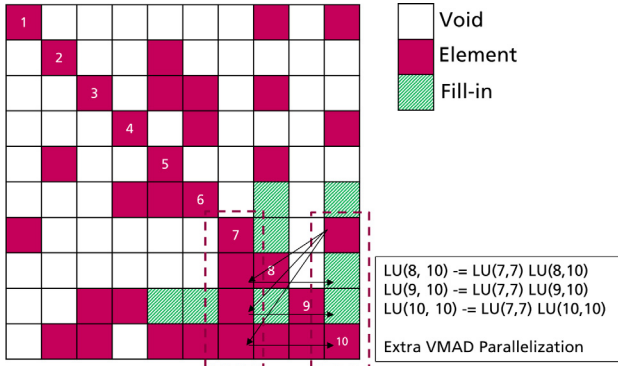


Fig. 6. Example LU matrix for LU Refactorization.

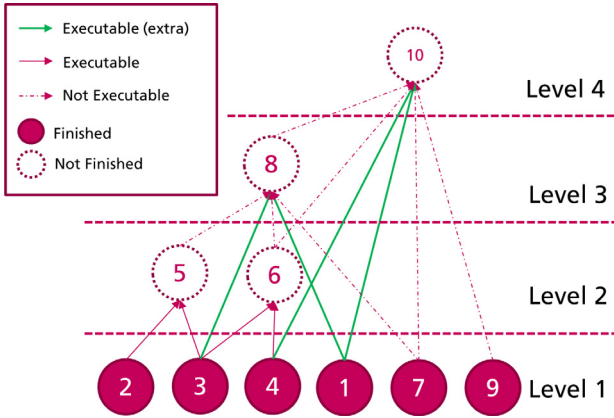


Fig. 7. Example scheduling graph for LU Refactorization.

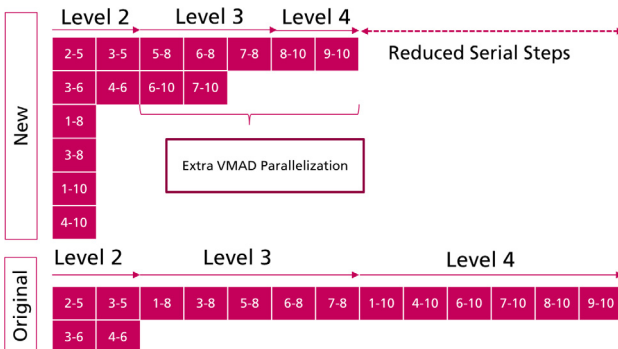


Fig. 8. Task scheduling on time line for LU Refactorization.

Algorithm 2 Sparse G-P refactorization algorithm with column working space

```

1: for tld in 0:Ntask do
2:   # do in parallel on CPU
3:   for col in 0:Ncol do
4:     # Copy column to working space
5:     x = A(:, col, tld)
6:     for row in URowlx(:, col) do
7:       # Sparse VMAD on column working space
8:       x(row + 1 :) = x(row) · L(:, row, tld)
9:     end for
10:    # Normalize L and update LU
11:    U(:, col, tld) = x(:, col)
12:    L(:, col, tld) = x(:, col + 1 :)/x(:, col)
13:  end for
14: end for

```

4.3. GPU batched LU refactorization and FS-BS

4.3.1. Theoretical analysis

Recent effort on LU factorization with GPU is presented in [24, 41], both works focus on accelerating single LU factorization on GPU, especially for large scaling matrix. For batched LU factorization, [20] presents a batched LU-factorization scheme with batch and column-level parallelization. It is essential to saturate the GPU memory bandwidth for LU refactorization and FS-BS to achieve good performance on GPU. Due to the underlying data dependency, not all columns can be factorized simultaneously. A directed acyclic graph (DAG) is used to describe and schedule the columns which can be finished at the same time [47]. The vertices represent the columns and the edges represent the operation between two columns. An example matrix and its DAG is shown in Figs. 6 and 7 respectively.

Considering the G-P algorithm with column-level parallelization proposed in [16], as the example of two standard grids shown in Fig. 5, at the beginning with high number of available columns per level, the GPU resource can be easily saturated even with small batch size. However, significant amount of FPs are located in the serial region, where only few columns can be factorized at the same time. Using NVIDIA GTX1080 graphics card as an example, the GPU has 20 SMs with each SM has 128 SPs. Assuming the batch size is 512, which equals to less than 1 active warps per SM, which means 75% of the SP remains idle when no extra parallelization is applied. Consequently, the memory-bandwidth cannot be efficiently utilized.

Inspired by the work [24], our work improves the batched LU-refactorization with fine-grained parallelization for serial levels, which improves the saturation of hardware significantly with small batch size. This improvement make greater sense on the

future applications with even more computational resources and higher memory-bandwidth available on new generations of GPUs.

4.3.2. Proposed algorithm on GPU

Algorithm 3 Multi-stage sparse G-P algorithm on CUDA

```

1: # DEFINE VMAD parallelization width as N
2: # Do in parallel on CUDA
3: # Find task (tld) with thread indexing
4: batchId = ThreadBlock.Idx
5: tldinBatch = Thread.Idx
6: tld = batchId · ThreadBlock.DimX + tldinBatch
7: # IF STAGE 1
8: col = AvailableColsinLevel(ThreadBlock.Idx)
9: # IF STAGE 2,3
10: col = LeftoverCols(ThreadBlock.Idx)
11: for row in URowIx(:, col) do
12:   # IF STAGE 2,3
13:   if row not in FinishedCols then
14:     break
15:   end if
16:   # IF STAGE 2,3
17:   if row in FinishedRowsOfCol(col) then
18:     # Skip finished rows
19:     continue
20:   end if
21:   # Sparse VMAD with direct indexing
22:   # IF STAGE 2 element-wise iteration in vector
23:   # IF STAGE 3 N-elements-wise iteration in vector
24:    $LU(row + 1 :, col, tld) =$ 
25:    $LU(row, col, tld) \cdot LU(row + 1 :, row, tld)$ 
26: end for
27: # Check column is finished
28: if row == URowIx(-1, col) then
29:   # Set flag on finished columns
30:   UpdateFinishedColStatus(col)
31:   # Normalize L
32:   # IF STAGE 2 element-wise iteration in vector
33:   # IF STAGE 3 N-elements-wise iteration in vector
34:    $LU(row + 1 :, col, tld) = LU(row, col, tld)$ 
35: else
36:   # IF STAGE 2,3
37:   # Memorize the processed row of the column
38:   UpdateFinishedRowsOfCol(row, col)
39: end if

```

A three-stage batched G-P algorithm is proposed for the GPU refactorization, which is given in Algorithm 3. LU denotes the working space for LU refactorization ($L + U - I$) including all the fill-ins predefined. The values in A needs to be copied into LU with the fill-ins initialized as 0.

In stage 1, with large number of columns available in each level, the columns are fully factorized in parallel. Each CUDA thread is responsible for one column in one task. In stage 2, besides the columns, which can be fully factorized, the viable operations for all the unfinished columns can be executed simultaneously (shown in Fig. 7 as green lines). In stage 3, the extra VMAD parallelization as shown in Fig. 6 could be applied with extra CUDA threads scheduled within the same CUDA *threadBlock*, since only threads within one *threadBlock* can be explicitly synchronized. The extra VMAD parallelization could use width of e.g. 4. In stage 2 and 3, on each level some columns can only be partially factorized, the already processed rows of each column are memorized, the finished columns of each level will be directly normalized.

Since the available columns in each parallel levels is known after the symbolic analysis step. As indicated in Fig. 5, the start

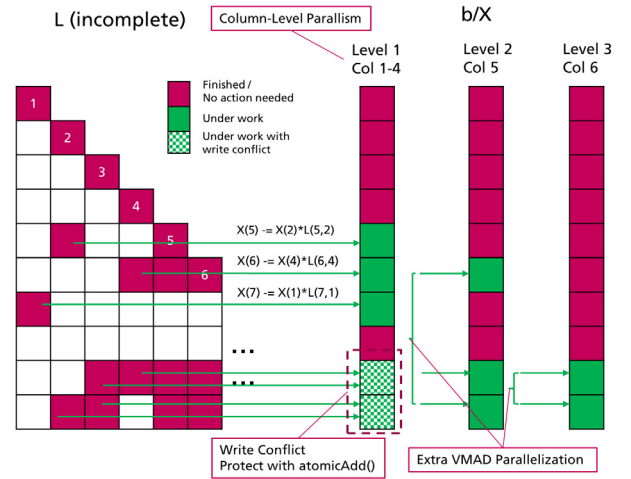


Fig. 9. Parallelized FS-BS.

of level 2 and level 3 is solely related to the available columns in each level. This parameter is related to the hardware. As manually tuned in our tests, to achieve good performance, the stage 2 can start when less than 32 columns are available in level and stage 3 starts when only 1 column is available.

For example matrix LU shown in Fig. 6, the level one in Fig. 7 corresponds to the stage 1 of Algorithm 3. After the first level is finished, tasks which belong to the columns in later levels can be executed in an earlier stage, which in turn increased the saturation rate and reduced the tasks in the serial levels (see Fig. 8). Level 2 corresponds to the stage 2 of the algorithm. In level 3, 4, when only one single column is available the extra VMAD parallelization is applied according to stage 3. In this case, the row 8, 9, 10 are assigned onto the *Thread.Idx*, thus VMAD can be performed more efficiently with suitable parallelization width instead of element-wise.

4.3.3. GPU forward substitution backward substitution

Fig. 9 shows the parallelization strategy of FS-BS with an incomplete L matrix. After the applying permutation matrix to the x , FS with L matrix can be performed on the same working space b/x . A dependency graph can be constructed to guide the parallel execution of multiple columns of L . As soon as the pivot element (upper element) of x corresponds to the column is finalized, this column can be executed to update the lower elements in x . When multiple threads try to update the same element, the barrier write function *atomicAdd()* protect from writing collision. The VMAD parallelization can be applied, when few columns are available on the same level. The same approach is applied for U .

5. Case studies and performance evaluation

The case study on CPU is performed on a Windows 10 PC with Intel i7-8700 CPU (6 physical cores and Hyper-Threading (HT) technology for 12 virtual cores) and 16 GB DDR4 RAM, with the prioritized Intel C++ Compiler V2020 and the Intel Math Kernel libraries, which is highly optimized for the vectorized computation. The code is compiled with O3 and forced using the AVX2 instruction set.

The case study on GPU is performed on a PC with Intel i7-8700k, 32 GB DDR4 RAM and 2x Nvidia GTX 1080-8GB GPUs. The PC is running on Ubuntu 18.04.4 LTS. The proposed approach is programmed in C++ and compiled with GCC V7.4 and CUDA V10.1 with O3. The main focus of the benchmarking is the duration of the PF solving and each subprocess. Because the

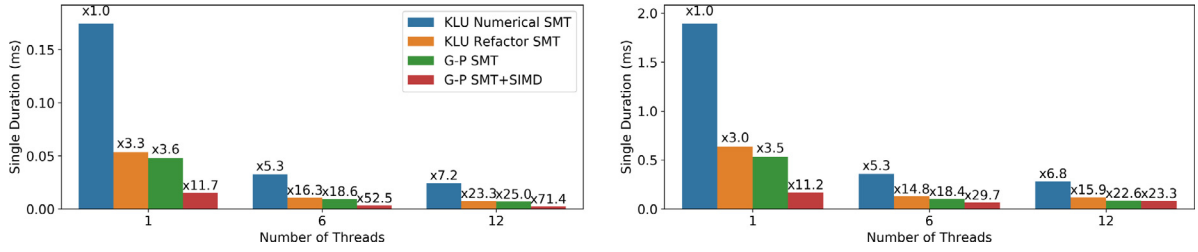


Fig. 10. Batched linear solver benchmark on CPU (left: IEEE case300, right: case2869pegase).

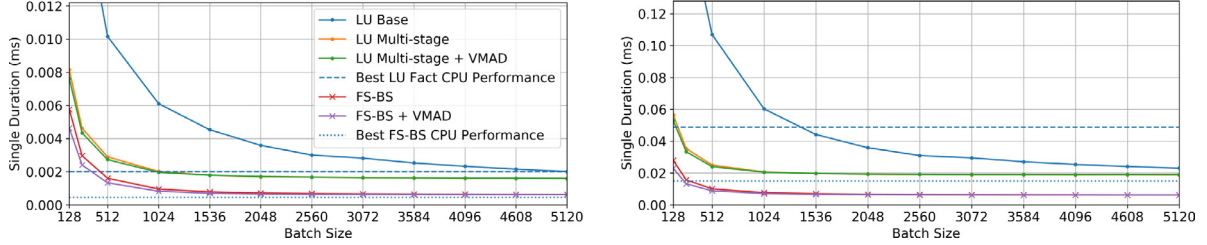


Fig. 11. Batched linear solver benchmark on GPU (left: IEEE case300, right: case2869pegase).

kernel execution in CUDA is initialized from CPU and executed on GPU asynchronously, to record the duration of each kernel, the test is performed with synchronization at the kernel (or multiple-kernels e.g. for LU-refactorization) exit.

On both test platforms, double precision float number is used, due to the high numerical stability requirement during the LU refactorization. The examples are tested with small-size grid “IEEE case300” with 300 buses and mid-size grid “case2869pegase” with 2869 buses available in pandapower. Both contain meshed Extra-High-Voltage (EHV) and High-Voltage (HV) voltage levels, IEEE case300 contains also MV and Low-Voltage (LV) buses.

5.1. Batched linear solver performance analysis

This section gives a performance evaluation of the aforementioned batched linear solver on CPU and GPU and the relative performance to KLU.

Fig. 10 shows the performance on CPU platform. On both test grids, a performance improvement of the implemented G-P algorithm can be observed over KLU against both numerical factorization and refactorization modes. Because it works directly on the already permuted A matrix, as the KLU takes the original matrix as input. With SMT parallelization, a performance improvement of $> \times 6$ can be observed for all the methods tested when increasing the number of threads, especially when below the number of physical cores (6). With further increasing the thread number, the HT technology helps further improve the saturation of the physical core, thus improve the overall performance. With the SMT+SIMD parallelization, further speed-ups can be observed. However, a slight performance decreasing can be observed on the mid-size grid under the G-P SMT+SIMD version, when increasing the threads number from 6 to 12, this behavior is caused by the reduced cache hitting rate, due to the large problem scaling. Overall, with the proposed method, on CPU platform, we achieved a good performance improvement of $\times 20$ – $\times 70$. The acceleration rate has a close relationship to the size of the problem (number of buses).

On the GPU platform, Fig. 11 shows the benchmarking result with different batch sizes, the best CPU results is used as baseline. It can be observed, that our approach of further fine-grained parallelization of the refactorization process leads to a earlier saturation of the GPU resource and achieved much better

performance when the batch size is small for both grids. With the extra VMAD parallelization with the 4x threads in stage 3, it improves the performance when the batch size is very small (≤ 512) for LU refactorization and FS-BS, slight difference can be observed when the GPU is fully saturated. When dealing with a large grid, the improvement is more significant comparing to the CPU counterpart due to the higher cache misses on CPU.

On the GPU test with a large grid (case9241pegase with 9241 buses), the simultaneous LU refactorization requires large working space on GPU memory, which leads to the fact that only batch size up to 2048 can be supported on the test platform. By setting the batch size to 2048, the average time of one LU-refactorization with proposed multi-stage method requires 0.138 ms while the base batch version requires 0.238 ms. The proposed method show also significant improvement in large grid.

5.2. Performance analysis on CPU

For the convenience of comparing the performance of each functions involved in NR PF, these can be categorized as following:

- FP Dominant (Float): Calculate NPM, Update $d\{P, Q\}/d\{V, |V|\}$
- Memory and FP (LU Total): LU Refactorization, FS-BS
- Memory Dominant (Memory): Permute $JM(A)$, Update V

The performance on CPU is evaluated with timing and Intel Vtune profiling tool for the memory access pattern analysis. Fig. 12 shows the time of each process of SMT parallelization and SMT+SIMD parallelization with different number of threads. For both small and mid-size grid, similar to the observation in batched linear solver, the performance increased almost linear at the beginning and only slightly after 6 threads. For the mid-size grid, with the last-level cache missing rate of the SMT+SIMD with 12 threads increased from 0.1% in total to 5% for “LU Total” and 25% for “memory” comparing to with only 4 threads. The random memory access pattern of “LU Total” and “memory” requires a frequent exchange between the cache and system DRAM, which due to its limited bandwidth actually, drags the overall performance back by increasing thread number. The SMT+SIMD version still outperforms 12 threaded SMT parallelization scheme at the thread number of 5. For the small grid, with the data fits in the

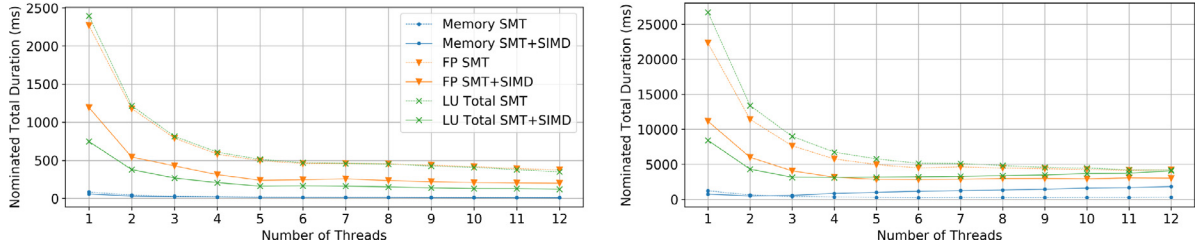


Fig. 12. Function benchmark results on CPU for the performance comparison over SMT and SMT+SIMD (left: IEEE case300, right: case2869pegase).

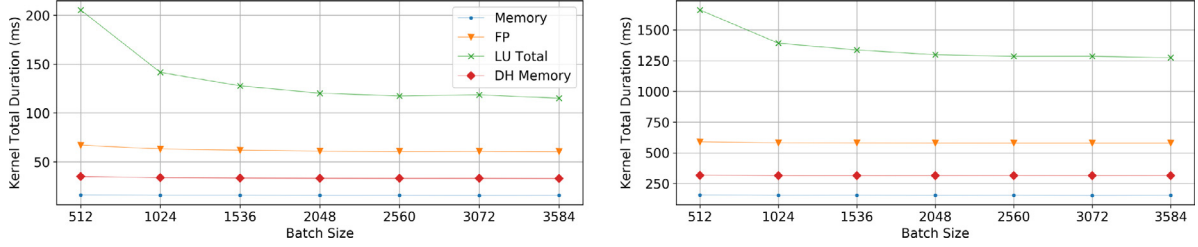


Fig. 13. Function benchmark results on GPU (left: IEEE case300, right: case2869pegase).

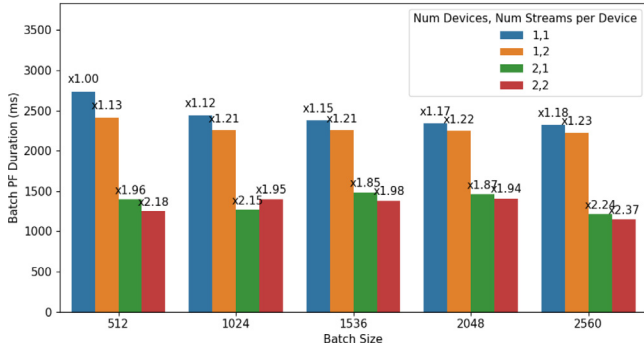


Fig. 14. Duration and speedups with batches and number of streams with case2869pegase.

cache well, the increase of threads number improves the overall performance.

Concluding, on CPU platform, increasing number of threads brings only benefit, when solving PF of a small scale grid. While with SIMD+SMT, the cache should be considered and the number of threads should be carefully chosen given the grid size and available hardware. On the test CPU for instance, the number of the physical cores (in this case 6) is a good first trial.

5.3. Performance analysis on GPU

The GPU version requires the extra memory transaction between host and device, which is labeled as “DH Memory”. Fig. 13 shows the function performance on GPU with one stream for both grids with 10,000 calculations. It can be observed that except for the “LU Total”, the other tasks can saturate the GPU resources with small batch size, thus it is insensitive to the change of the batch size for both grids. Comparing to the best CPU performance, the “FP” function achieved the average improvement among x5–6.

Fig. 14 shows the improvement with the usage of CUDA concurrency with stream and multi GPUs on different batch sizes with case2869pegase. The test case with batch size 512 and 1 stream is used as base line scenario. With the multiple streams on one GPU, an improvement due to the hiding the memory transaction between host and device can be observed with all batch sizes.

When the batch size is small, the improvement is higher which is related to the potential of kernel overlapping. When two GPUs are available, an acceleration of around factor x2 can be observed. However, due to the imbalanced computation load distribution among GPUs and streams. With specific stream/GPU undertakes more tasks than the others, the improvement of the overall execution time varies. Using batch size 2048 as an example, when executing 10,000 PFs, 4x2048 tasks are equally distributed, while one GPU/Stream have to execute the 1808 leftovers.

5.4. Benchmarking result of the integrated parallel PF-solver

The benchmarking result of running 10,000 PFs of multiple grids with same Y_{bus} is listed in Table 1. The parallel PF-solver is integrated in pandapower. Besides case2869pegase and IEEE case300, further standard grids “IEEE case118”, “case1354pegase” and “case9241pegase” available in pandapower are utilized to evaluate the performance under different grid dimensions. As well as three grids from the SimBench open-source dataset [48], which was recreated to represent the characteristics of real German grid and contains realistic time series data for loads and DERs, are used for benchmark. The dataset contains yearly time series of 15-minutes resolution (35040 total time steps), the first 10,000 time steps are used. With multiple performance optimizations especially the creation of JM, pandapower gives already better performance comparing to MATPOWER [9,39], thus the baseline case is performed with PF-function of pandapower named “newtonpf”, which is not parallelized and used SuperLU as linear system solver.

Fig. 15 shows the acceleration factor with regard to the number of buses in the grid. The acceleration factor is calculated by $T_{pandapower}/T_{case}$. It would be fair for the evaluation of the gained acceleration with the comparison against baseline, since the required NR iterations and the grid topology are the same in pandapower and in the proposed approach. As can also be seen in Table 1, with the increase of grid dimension, the acceleration on the GPU is more significant as on CPU, due to the high FP capability and memory bandwidth of GPU. On large grids, e.g. case9241pegase, the GPU version is x4 times faster than the best CPU performance. To be noticed by case9241pegase, due to the large requirements of GPU memory, batch size larger than 2048 failed in the benchmarking. The effect of acceleration by using SMT+SIMD is also decreasing comparing to SMT due to the

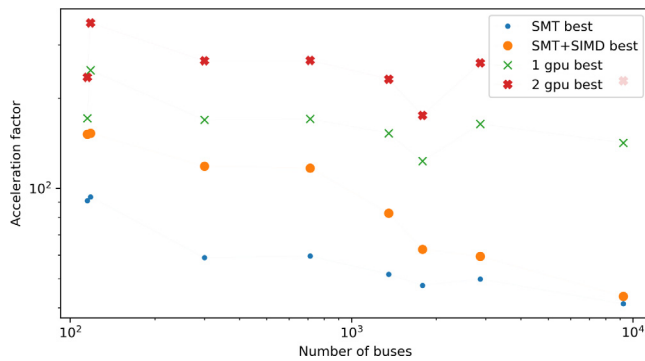


Fig. 15. Acceleration factor of proposed approach comparing to baseline case pandapower including initialization.

forementioned cache issue. On small size grid, because of the well usage of CPU cache, the performance of CPU is satisfying.

From Table 1, it can be further observed, that the grid dimension has direct impact on the one-time initialization time, which is according to Fig. 4 applied on both CPU and GPU cases. The one-time initialization takes significant amount of time, since most of the code is executed in Python environment, which can be further optimized e.g. with just-in time (JIT) compilation or C/C++ integration.

In the test considering the number of PF calculation, when increasing the number of calculation from 100 to 10,000, the number of calculation has few impact on the average time of solving single PF on CPU SMT and SMT+SIMD. The GPU version, due to the aforementioned saturation issue, the performance is improved with the increase of the number of calculation. After the saturation point of around 2000 calculations is reached, the average time of solving PF is almost constant.

6. Conclusion

In this paper, an approach for the parallel solving of many power flows with CPU/GPU is proposed. When the grid admittance matrices share the same sparsity pattern, the Newton-Raphson power flows can be efficiently solved in parallel. The performance is evaluated in detail with multiple test cases covering small, mid-size and large grids.

Impressive acceleration (more than 100x over single threaded open-source tool pandapower and at least 16x can be expected if pandapower can be perfectly parallelized on 6 cores) was achieved with CPU/GPU parallelization. The performance of the fast parallel power flow solver originated mainly from the following points:

- Avoidance of repetitive work (sparse matrix indexing initialization, pre-ordering, lookup creation, etc.),
- Reduction of computational overhead with LU-refactorization,
- Hardware-specific optimization and parallelization strategies.

In detail, on CPU platform, the power flows can be accelerated with less effort with SMT parallelization. With SMT+SIMD parallelization, the acceleration effect is highly dependent to the problem scaling. On small grids, a further speed-up of x2–3 comparing to SMT parallelization can be expected. On the GPU platform, with the batch operation on sparse matrix, the high FP capability and high memory bandwidth can be effectively saturated. Comparing to the CPU counterparts, the computing-bounding functions are accelerated significantly, while the memory-bounding functions depend highly on the problem scaling.

The outstanding performance of the proposed parallel power flow solver shows promising application in the real-time grid operation in order to allow the consideration of uncertainties. The innovative researches in the data-driven machine-learning methods in power systems can be great benefited. Even more potential can be exploited with the application of the proposed solver on a high-performance computing clusters with multiple CPUs and GPUs.

CRediT authorship contribution statement

Zhenqi Wang: Conceptualization, Methodology, Software, Writing - original draft, Writing - review & editing. **Sebastian Wende-von Berg:** Project administration, Conceptualization, Writing - review & editing. **Martin Braun:** Supervision, Conceptualization, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank Florian Schäfer and Dr. Alexander Scheidler for their suggestions to improve the quality of this paper. The work was supported by the European Union's Horizon 2020 research and innovation programme within the project EU-SysFlex under grant agreement No. 773505.

References

- [1] H. Yu, C.Y. Chung, K.P. Wong, H.W. Lee, J.H. Zhang, Probabilistic load flow evaluation with hybrid latin hypercube sampling and cholesky decomposition, *IEEE Trans. Power Syst.* 24 (2) (2009) 661–667, <http://dx.doi.org/10.1109/TPWRS.2009.2016589>.
- [2] C.-L. Su, Probabilistic load-flow computation using point estimate method, *IEEE Trans. Power Syst.* 20 (4) (2005) 1843–1851, <http://dx.doi.org/10.1109/TPWRS.2005.857921>.
- [3] J. Usaola, Probabilistic load flow with correlated wind power injections, *Electr. Power Syst. Res.* 80 (5) (2010) 528–536, <http://dx.doi.org/10.1016/j.epsr.2009.10.023>.
- [4] F. Schäfer, J.-H. Menke, M. Braun, Contingency analysis of power systems with artificial neural networks, in: 2018 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids, SmartGridComm, IEEE, 2018, pp. 1–6, <http://dx.doi.org/10.1109/SmartGridComm.2018.8587482>.
- [5] K.R. Mestav, J. Luengo-Rozas, L. Tong, Bayesian state estimation for unobservable distribution systems via deep learning, *IEEE Trans. Power Syst.* 34 (6) (2019) 4910–4920, <http://dx.doi.org/10.1109/TPWRS.2019.2919157>.
- [6] Y. Liu, N. Zhang, Y. Wang, J. Yang, C. Kang, Data-driven power flow linearization: A regression approach, *IEEE Trans. Smart Grid* 10 (3) (2019) 2569–2580, <http://dx.doi.org/10.1109/TSG.2018.2805169>.
- [7] J.-H. Menke, N. Bornhorst, M. Braun, Distribution system monitoring for smart power grids with distributed generation using artificial neural networks, *Int. J. Electr. Power Energy Syst.* 113 (2019) 472–480, <http://dx.doi.org/10.1016/j.ijepes.2019.05.057>.
- [8] R.D. Zimmerman, C.E. Murillo-Sanchez, R.J. Thomas, MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education, *IEEE Trans. Power Syst.* 26 (1) (2011) 12–19, <http://dx.doi.org/10.1109/TPWRS.2010.2051168>.
- [9] L. Thurner, A. Scheidler, F. Schäfer, J.-H. Menke, J. Dollichon, F. Meier, S. Meinecke, M. Braun, Pandapower—An open-source python tool for convenient modeling, analysis, and optimization of electric power systems, *IEEE Trans. Power Syst.* 33 (6) (2018) 6510–6521, <http://dx.doi.org/10.1109/TPWRS.2018.2829021>.
- [10] I. Araújo, V. Tadaiesky, D. Cardoso, Y. Fukuyama, Á. Santana, Simultaneous parallel power flow calculations using hybrid CPU-GPU approach, *Int. J. Electr. Power Energy Syst.* 105 (2019) 229–236, <http://dx.doi.org/10.1016/j.ijepes.2018.08.033>.

- [11] C. Guo, B. Jiang, H. Yuan, Z. Yang, L. Wang, S. Ren, Performance comparisons of parallel power flow solvers on GPU system, in: 2012 IEEE International Conference on Embedded and Real-Time Computing Systems and Applications, IEEE, 2012, pp. 232–239, <http://dx.doi.org/10.1109/RTCSA.2012.36>.
- [12] X. Su, C. He, T. Liu, L. Wu, Full parallel power flow solution: A GPU-CPU-based vectorization parallelization and sparse techniques for Newton-Raphson implementation, IEEE Trans. Smart Grid 11 (3) (2020) 1833–1844, <http://dx.doi.org/10.1109/TSG.2019.2943746>.
- [13] X. Li, F. Li, H. Yuan, H. Cui, Q. Hu, GPU-Based fast decoupled power flow with preconditioned iterative solver and inexact Newton method, IEEE Trans. Power Syst. 32 (4) (2017) 2695–2703, <http://dx.doi.org/10.1109/TPWRS.2016.2618889>.
- [14] V. Roberge, M. Tarbouchi, F. Okou, Parallel power flow on graphics processing units for concurrent evaluation of many networks, IEEE Trans. Smart Grid 8 (4) (2017) 1639–1648, <http://dx.doi.org/10.1109/TSG.2015.2496298>.
- [15] G. Zhou, Y. Feng, R. Bo, L. Chien, X. Zhang, Y. Lang, Y. Jia, Z. Chen, GPU-accelerated batch-ACPF solution for N-1 static security analysis, IEEE Trans. Smart Grid 8 (3) (2017) 1406–1416, <http://dx.doi.org/10.1109/TSG.2016.2600587>.
- [16] G. Zhou, R. Bo, L. Chien, X. Zhang, S. Yang, D. Su, GPU-accelerated algorithm for online probabilistic power flow, IEEE Trans. Power Syst. 33 (1) (2018) 1132–1135, <http://dx.doi.org/10.1109/TPWRS.2017.2756339>.
- [17] M. Abdelaziz, GPU-OpenCL accelerated probabilistic power flow analysis using Monte-Carlo simulation, Electr. Power Syst. Res. 147 (2017) 70–72, <http://dx.doi.org/10.1016/j.epsr.2017.02.022>.
- [18] M.M.A. Abdelaziz, Opencl-accelerated probabilistic power flow for active distribution networks, IEEE Trans. Sustain. Energy 9 (3) (2018) 1255–1264, <http://dx.doi.org/10.1109/TSTE.2017.2781148>.
- [19] S. Huang, V. Dinavahi, Real-time contingency analysis on massively parallel architectures with compensation method, IEEE Access 6 (2018) 44519–44530, <http://dx.doi.org/10.1109/ACCESS.2018.2864757>.
- [20] G. Zhou, R. Bo, L. Chien, X. Zhang, F. Shi, C. Xu, Y. Feng, GPU-based batch LU-factorization solver for concurrent analysis of massive power flows, IEEE Trans. Power Syst. 32 (6) (2017) 4975–4977, <http://dx.doi.org/10.1109/TPWRS.2017.2662322>.
- [21] S. Huang, V. Dinavahi, Fast batched solution for real-time optimal power flow with penetration of renewable energy, IEEE Access 6 (2018) 13898–13910, <http://dx.doi.org/10.1109/ACCESS.2018.2812084>.
- [22] A. Fog, The Microarchitecture of Intel, AMD and VIA CPUs: An Optimization Guide for Assembly Programmers and Compiler Maker, Technical University of Denmark, <https://www.agner.org/optimize/microarchitecture.pdf>.
- [23] T.A. Davis, E.P. Natarajan, Algorithm 907: KLU, a direct sparse solver for circuit simulation problems, ACM Trans. Math. Software 37 (3) (2010) 1–17, <http://dx.doi.org/10.1145/1824801.1824814>.
- [24] X. Chen, L. Ren, Y. Wang, H. Yang, GPU-accelerated sparse LU factorization for circuit simulation with performance modeling, IEEE Trans. Parallel Distrib. Syst. 26 (3) (2015) 786–795, <http://dx.doi.org/10.1109/TPDS.2014.2312199>.
- [25] P. Gepner, Using AVX2 instruction set to increase performance of high performance computing code, Comput. Inform. 36 (5) (2017) 1001–1018.
- [26] E. Khartchenko, Vectorization: Performance with quantifi, Intel, <https://software.intel.com/content/www/us/en/develop/articles/vectorization-a-key-tool-to-improve-performance-on-modern-cpus.html>.
- [27] J.D. Owens, M. Houston, D. Luebke, S. Green, J.E. Stone, J.C. Phillips, GPU Computing, Proc. IEEE 96 (5) (2008) 879–899, <http://dx.doi.org/10.1109/JPROC.2008.917757>.
- [28] NVIDIA, CUDA C++ programming guide.
- [29] L. Dagum, R. Menon, Openmp: an industry standard API for shared-memory programming, IEEE Comput. Sci. Eng. 5 (1) (1998) 46–55, <http://dx.doi.org/10.1109/99.660313>.
- [30] D.M. Tullsen, S.J. Eggers, H.M. Levy, Simultaneous multithreading: Maximizing on-chip parallelism, in: Proceedings 22nd Annual International Symposium on Computer Architecture, 1995, pp. 392–403.
- [31] OpenMP Architecture Review Board, OpenMP application program interface: Version 4.0. <https://www.openmp.org/wp-content/uploads/OpenMP4.0.0.pdf>.
- [32] Y. Zhang, J.D. Owens, A quantitative performance analysis model for GPU architectures, in: 2011 IEEE 17th International Symposium on High Performance Computer Architecture, IEEE, 2011, pp. 382–393, <http://dx.doi.org/10.1109/HPCA.2011.5749745>.
- [33] V. Volkov, Understanding Latency Hiding on GPUs (Ph.D. thesis), EECS Department, University of California, Berkeley, 2016, <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-143.html>.
- [34] NVIDIA, CUDA C++ best practices guide, <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html>.
- [35] J. Grainger, W. Stevenson, Power System Analysis, McGraw-Hill, New York and London, 1994.
- [36] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D.S. Seljebotn, K. Smith, Cython: The best of both worlds, Comput. Sci. Eng. 13 (2) (2011) 31–39, <http://dx.doi.org/10.1109/MCSE.2010.118>.
- [37] W. McKinney, Data structures for statistical computing in python, in: Proceedings of the 9th Python in Science Conference, SciPy, 2010, pp. 56–61, <http://dx.doi.org/10.25080/Majora-92bf1922-00a>.
- [38] C.R. Harris, K.J. Millman, S.J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N.J. Smith, R. Kern, M. Picus, S. Hoyer, M.H. van Kerkwijk, M. Brett, A. Haldane, J.F. Del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T.E. Oliphant, Array programming with NumPy, Nature 585 (7825) (2020) 357–362, <http://dx.doi.org/10.1038/s41586-020-2649-2>.
- [39] F. Schafer, M. Braun, An efficient open-source implementation to compute the Jacobian matrix for the Newton-Raphson power flow algorithm, <http://dx.doi.org/10.1109/ISGTEurope.2018.8571471>.
- [40] X.S. Li, An overview of SuperLU, ACM Trans. Math. Software 31 (3) (2005) 302–325, <http://dx.doi.org/10.1145/1089014.1089017>.
- [41] S. Peng, S.X.D. Tan, GLU3.0: Fast GPU-based parallel sparse LU factorization for circuit simulation, URL <http://arxiv.org/pdf/1908.00204v3>.
- [42] L. Razik, L. Schumacher, A. Monti, A. Guironnet, G. Bureau, A comparative analysis of LU decomposition methods for power system simulations, pp. 1–6, <http://dx.doi.org/10.1109/PTC.2019.8810616>.
- [43] P.R. Amestoy, Enseeiht-Irit, T.A. Davis, I.S. Duff, Algorithm 837: AMD, an approximate minimum degree ordering algorithm, ACM Trans. Math. Software 30 (3) (2004) 381–388, <http://dx.doi.org/10.1145/1024074.1024081>.
- [44] I. Kocar, J. Mahseredjian, U. Karaagac, G. Soykan, O. Saad, Multiphase load-flow solution for large-scale distribution systems using MANA, IEEE Trans. Power Deliv. 29 (2) (2014) 908–915, <http://dx.doi.org/10.1109/TPWRD.2013.2279218>.
- [45] J.R. Gilbert, T. Peierls, Sparse Partial Pivoting in Time Proportional to Arithmetic Operations: TR 86-783, Cornell University.
- [46] X. Chen, Y. Wang, H. Yang, NICSLU: An adaptive sparse matrix solver for parallel circuit simulation, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 32 (2) (2013) 261–274, <http://dx.doi.org/10.1109/TCAD.2012.2217964>.
- [47] T. Nechma, M. Zwolinski, Parallel sparse matrix solution for circuit simulation on FPGAs, IEEE Trans. Comput. 64 (4) (2015) 1090–1103, <http://dx.doi.org/10.1109/TC.2014.2308202>.
- [48] S. Meinecke, D. Sarajlić, S.R. Drauz, A. Klettke, L.-P. Lauen, C. Rehtanz, A. Moser, M. Braun, Simbench—A benchmark dataset of electric power systems to compare innovative solutions based on power flow analysis, Energies 13 (12) (2020) 3290, <http://dx.doi.org/10.3390/en13123290>.